Experiment – 1 a: TypeScript

Name of Student	**Manorath Ital**
Class Roll No	**19**
D.O.P.	**28-01-2025**
D.O.S.	**11-02-2025**
Sign and Grade	

1. **Aim:** Write a simple TypeScript program using basic data types (number, string, boolean) and operators.

2. **Problem Statement:**

   a. Create a calculator in TypeScript that uses basic operations like addition, subtraction, multiplication, and division. It also gracefully handles invalid operations and division by zero..

   b. Design a Student Result database management system using TypeScript.

```
// Step 1: Declare basic data types

const studentName: string = "John Doe";

const subject1: number = 45;

const subject2: number = 38;

const subject3: number = 50;

// Step 2: Calculate the average marks

const totalMarks: number = subject1 + subject2 + subject3;

const averageMarks: number = totalMarks / 3;

//          Step 3: Determine if the student has
passed or failed const isPassed: boolean =
averageMarks >= 40;
//
```

```
// Step 4: Display the result

console.log(Student Name: ${studentName});

console.log(Average Marks: ${averageMarks});

console.log(Result: ${isPassed ? "Passed" : "Failed"});
```

Different Data Types in TypeScript & Type Annotations

TypeScript provides a variety of data types that help ensure safer and more readable code. These include:

- **Primitive types** like `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, and `bigint`.
- **Special types** such as `any` (accepts any type), `unknown` (a safer alternative to `any`), `void` (used for functions that don't return a value), and `never` (used for functions that never return, like those throwing errors).
- **Complex types** including objects, arrays, and tuples, which are fixed-length collections that can hold multiple types.

Type Annotations in TypeScript are used to declare the expected type of variables, function parameters, and return values. This helps the compiler catch type-related errors before runtime and improves clarity in the code.

How TypeScript Files Are Compiled

To compile TypeScript:

1. First, install TypeScript globally using `npm install -g typescript`.
2. Use the `tsc` command followed by the filename to compile the code (e.g., `tsc file.ts`).
3. To automatically compile on file changes, use `tsc --watch`.
4. A `tsconfig.json` file can be used for custom project configurations and compiler options.

JavaScript vs TypeScript – Key Differences

Feature	JavaScript	TypeScript
Typing	Dynamically typed	Statically typed (with optional type annotations)
Compilation	Directly run in browser	Needs to be compiled to JavaScript
Type Checking	No type checks at compile time	Compile-time type validation
ES Features Support	Works with ES5 (older JS)	Supports modern features like async/await, decorators, etc.
Error Detection	Caught only during runtime	Many errors caught before execution
OOP Support	Limited, prototype-based	Full object-oriented support (classes, interfaces, etc.)
Interoperability	Works natively in browsers and Node.js	Needs compilation to work in JS environments

Inheritance in JavaScript vs TypeScript

JavaScript implements inheritance using prototypes, where objects inherit from other objects. TypeScript, on the other hand, uses class-based inheritance, offering structured features such as access modifiers and the ability to implement interfaces for enforcing consistency.

Generics – Making Code More Flexible

Generics make functions and classes more adaptable by allowing them to work with various types without compromising type safety. Unlike using `any`, which disables type checking, generics retain type information, ensuring the code behaves as expected.

Classes vs Interfaces in TypeScript

Classes are templates used to create objects and can include properties, methods, and constructors. They also support inheritance and can implement interfaces.

Interfaces, on the other hand, only define the structure of data — such as the shape of an object or what methods and properties it should have — without providing any logic. Interfaces are commonly used to ensure that classes or objects follow a specific pattern.

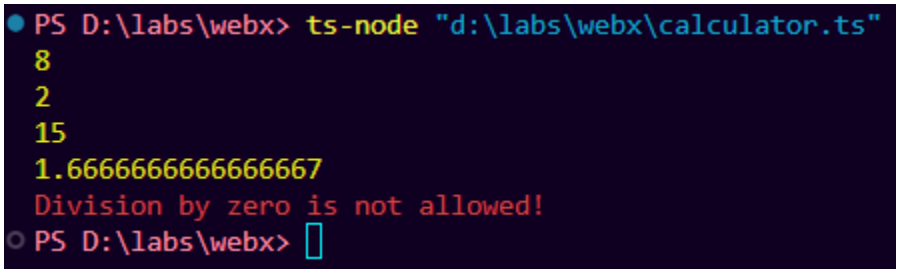
GitHub Link: - https://github.com/MANO-RATH/WEBX-EXP1.git

Input:

```
function calculator(a: number, b: number, operator: string): number | never {
    switch (operator) {
        case "+":
            return a + b;
        case "-":
            return a - b;
        case "*":
            return a * b;
        case "/":
            if (b === 0) {
                throw new Error("Division by zero is not allowed!");
            }
            return a / b;
        default:
            throw new Error(`Invalid operator: '${operator}'. Use +, -, *, or /.`);
    }
}

try {
    console.log(calculator(5, 3, "+"));
    console.log(calculator(5, 3, "-"));
    console.log(calculator(5, 3, "*"));
    console.log(calculator(5, 3, "/"));
    console.log(calculator(7, 0, "/"));
    console.log(calculator(10, 2, "%"));
} catch (error) {
```

```
    console.error((error as Error).message);
}
```

Output:

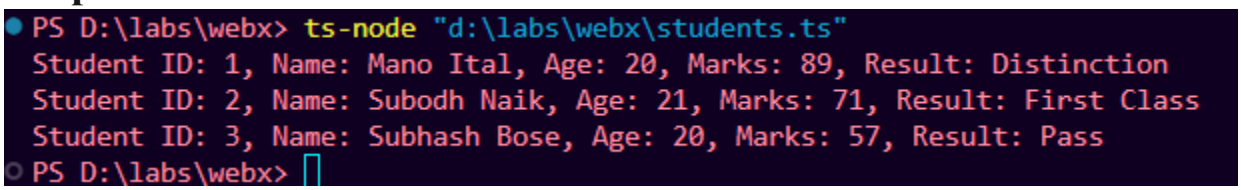


Input:

```
type Student = {
    id: number;
    name: string;
    surname: string;
    age: number;
    marks: number;
};

const students: Student[] = [
    { id: 1, name: "Mano", surname: "Ital", age: 20, marks: 89 },
    { id: 2, name: "Subodh", surname: "Naik", age: 21, marks: 71 },
    { id: 3, name: "Subhash", surname: "Bose", age: 20, marks: 57 },

];

const determineResult = (marks: number): string => {
    if (marks < 40) return "Fail";
    if (marks < 60) return "Pass";
    if (marks < 75) return "First Class";
```

```
    return "Distinction";
};


students.forEach((student) => {
    const result = determineResult(student.marks);
    console.log(
        `Student ID: ${student.id}, Name: ${student.name} ${student.surname}, Age: ${student.age}, Marks: ${student.marks}, Result: ${result}`,
    );
});
```

Output:



Conclusion:-

The TypeScript programs demonstrate the use of basic data types such as `number`, `string`, and `boolean`, along with operators in practical scenarios. The calculator effectively performs arithmetic operations while handling exceptions like division by zero. Similarly, the student result management system processes marks, calculates averages, and determines pass/fail status based on a set threshold.

These implementations highlight TypeScript's strong type safety and logical capabilities, ensuring code reliability, maintainability, and clarity in real-world applications.